LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

INHERITANCE (PEWARISAN CLASS)



Identitas Mahasiswa

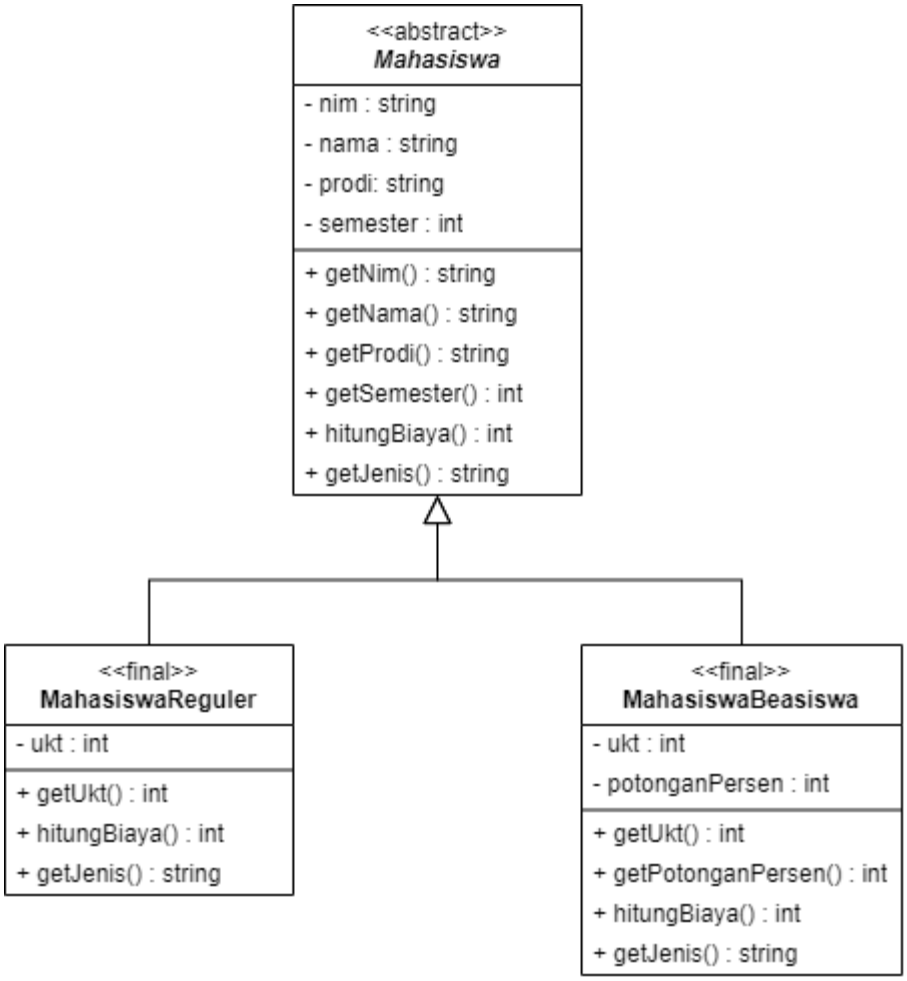
Nama	: Diandras Rio Triasmoro
NIM	: 2530521062
Kelas	: 3 TI C

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS MUHAMMADIYAH SUKABUMI
2026

1. Tujuan dan Ringkasan Konsep

1. Menjelaskan hubungan is-a dan membedakannya dari has-a (komposisi).
2. Menggunakan extends, parent::__construct(), dan method overriding.
3. Memahami perbedaan akses public, protected, dan private pada pewarisan.
4. Membangun CRUD mahasiswa yang menyimpan dua jenis objek (Reguler dan Beasiswa) tanpa menggandakan logika dasar.
5. Memahami bahwa PHP hanya mendukung single inheritance dan trait bukan multiple inheritance.

2. Diagram Class dan Hubungan Parent–Child



Gambar 1 Diagram Class

Penjelasan hubungan parent-child :

1. MahasiswaReguler dan MahasiswaBeasiswa adalah turunan dari Mahasiswa (hubungan is-a). Keduanya sama-sama mahasiswa dengan NIM, nama, prodi, dan semester.

2. Di parent : data identitas (nim, nama, prodi, semester), validasinya, dan getter nya. Karena itu tidak ada kode identitas yang diulang di child.
3. Di child : data khusus (ukt, potonganPersen) dan rumus biaya masing-masing.
4. Mahasiswa dibuat abstract karena mahasiswa umum belum punya rumus biaya yang lengkap. Child wajib mengimplementasikan hitungBiaya() dan getJenis().
5. Kedua child bersifat final karena tidak dirancang untuk diturunkan lagi.

Alasan pemilihan desain

1. Menghindari duplikasi kode: validasi dan data identitas cukup ditulis sekali di parent.
2. Perbedaan perilaku (rumus biaya) dipisahkan di child lewat overriding, sehingga mudah dirawat dan dikembangkan.
3. Repository DataMahasiswa cukup menerima tipe Mahasiswa, tanpa percabangan if per jenis.
4. Enkapsulasi tetap terjaga karena property parent private dan diakses lewat getter.

Perbandingan private vs protected

1. private $nim hanya bisa diakses langsung oleh Mahasiswa. Child harus memakai getNim(). Ini menjaga data identitas agar tidak diubah sembarangan oleh child.
2. protected $kategori bisa diakses langsung oleh parent dan child, tetapi tetap tidak bisa diakses dari luar class.
3. Pada studi kasus ini, private dipilih untuk data identitas karena validasinya ada di constructor parent. Jika child boleh mengubahnya langsung, validasi bisa terlewati. protected sebaiknya dipakai selektif, hanya jika child memang perlu akses langsung.

3. Potongan Kode Penting

A. extends

```php
<?php
abstract class Mahasiswa {
    // property identitas (private) + constructor + getter
    abstract public function hitungBiaya(): int;
    abstract public function getJenis(): string;
}

final class MahasiswaReguler extends Mahasiswa {
    private int $ukt;
    // ...
}
```

```php
final class MahasiswaBeasiswa extends Mahasiswa {
    private int $ukt;
    private int $potonganPersen;
    // ...
}
```

Penjelasan: Kata kunci extends menyatakan bahwa MahasiswaReguler dan MahasiswaBeasiswa adalah turunan Mahasiswa (hubungan is-a). Keduanya otomatis mewarisi method public getNim(), getNama(), getProdi(), dan getSemester() tanpa menulis ulang kodenya.

B. parent::__construct()

```php
public function __construct(string $nim, string $nama,
        string $prodi, int $semester, int $ukt, int
$potonganPersen) {
    parent::__construct($nim, $nama, $prodi, $semester);
    if ($ukt < 0 || $potonganPersen < 0 || $potonganPersen > 100) {
        throw new InvalidArgumentException('UKT/potongan tidak
valid');
    }
    $this->ukt = $ukt;
    $this->potonganPersen = $potonganPersen;
}
```

Penjelasan: Karena child punya constructor sendiri, constructor parent tidak dipanggil otomatis. parent::__construct() ditulis eksplisit agar NIM, nama, prodi, dan semester diinisialisasi dan divalidasi oleh parent. Setelah itu, child memvalidasi dan mengisi data khususnya (UKT dan potongan). Kalau baris ini dihapus, property identitas tidak terisi.

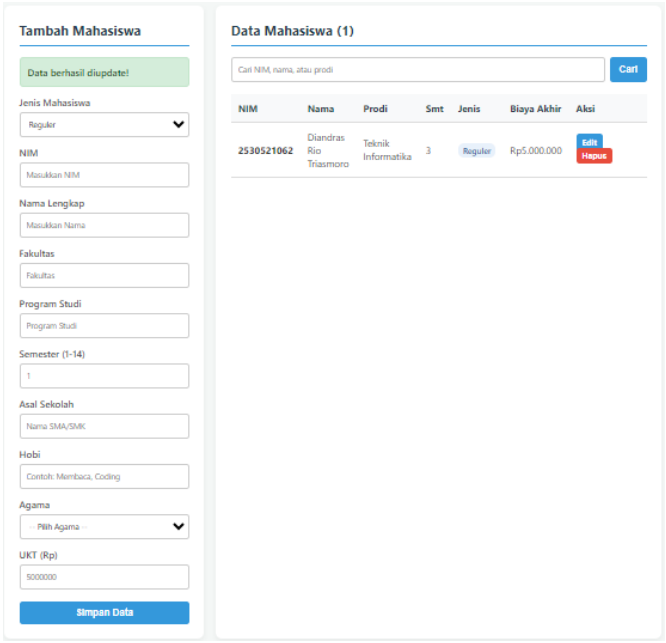
C. overriding

```php
// MahasiswaReguler
public function hitungBiaya(): int {
    return $this->ukt;
}

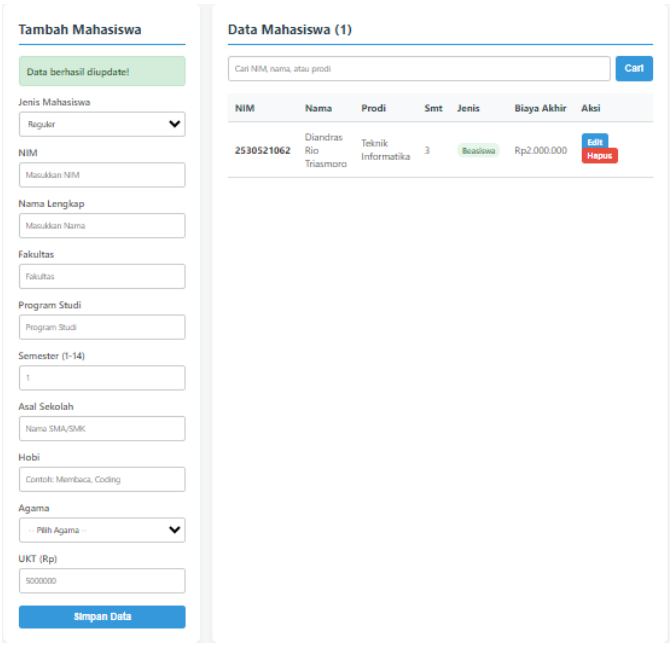
// MahasiswaBeasiswa
public function hitungBiaya(): int {
    return intdiv($this->ukt * (100 - $this->potonganPersen), 100);
```

Penjelasan: Method hitungBiaya() dideklarasikan abstract di parent, lalu diimplementasikan dengan rumus berbeda di tiap child. Reguler membayar UKT penuh, sedangkan Beasiswa membayar UKT dikurangi persentase potongan. Contoh: UKT 5.000.000 dengan potongan 60% menghasilkan biaya 2.000.000. Pemanggil cukup menulis $m->hitungBiaya() tanpa mengecek jenis objeknya.

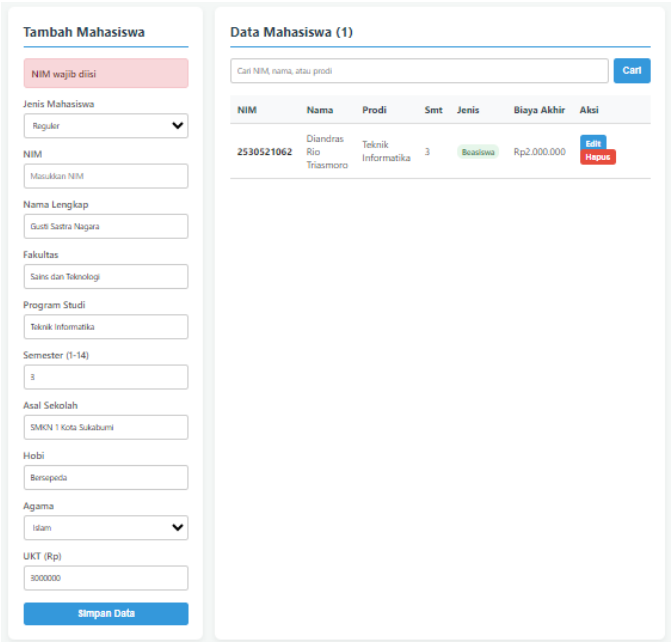
4. Screenshot CRUD untuk Reguler dan Beasiswa (Tugas 3)



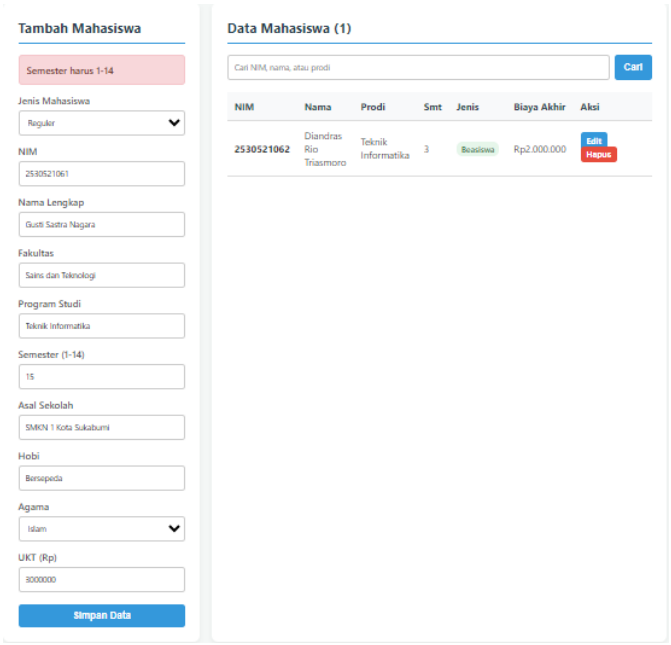
Tambah Reguler UKT 5.000.000



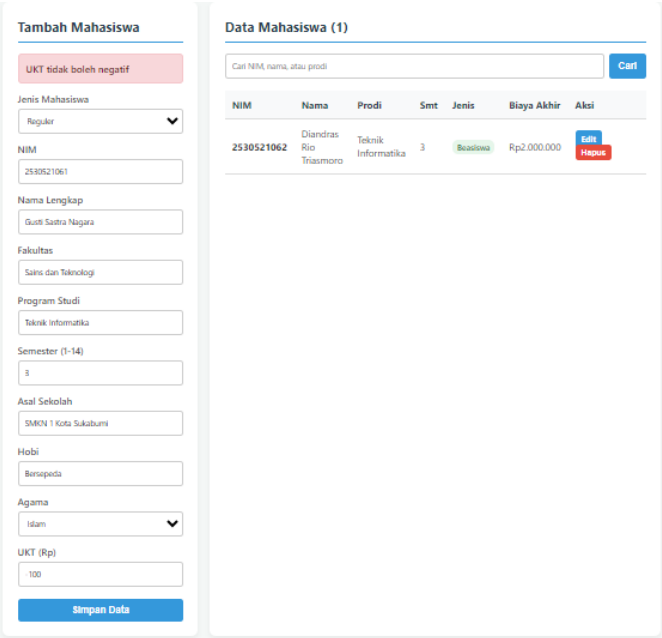
Tambah Beasiswa UKT 5.000.000; potongan 60%



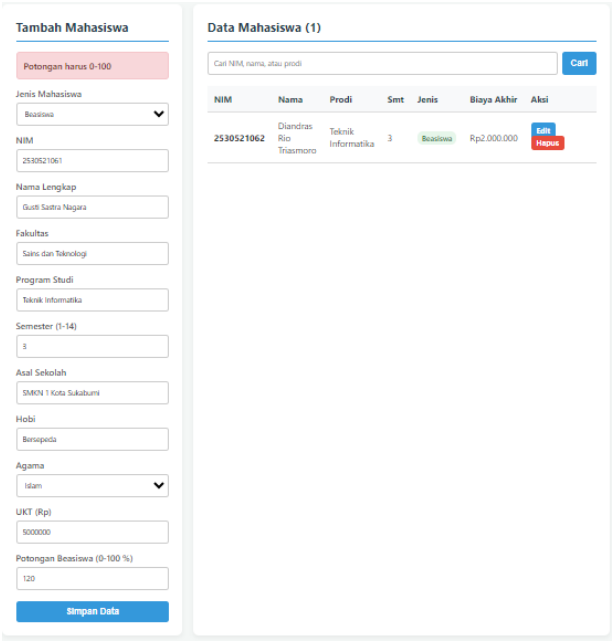
NIM kosong



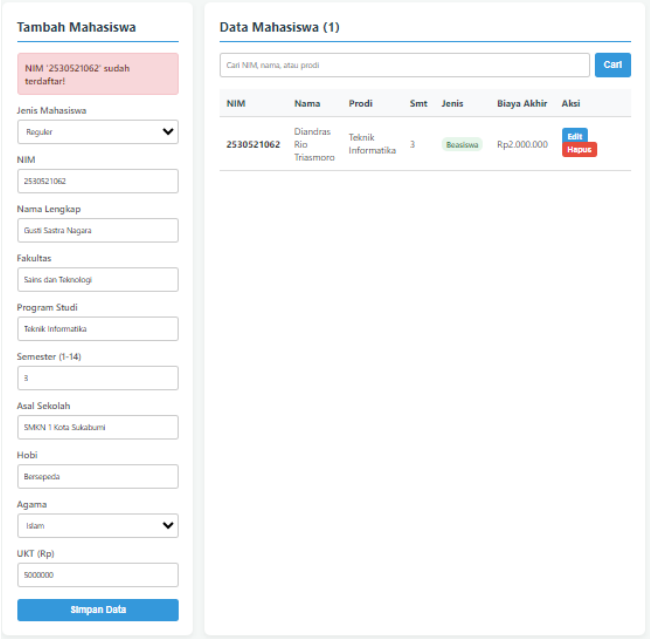
Semester 0 atau 15



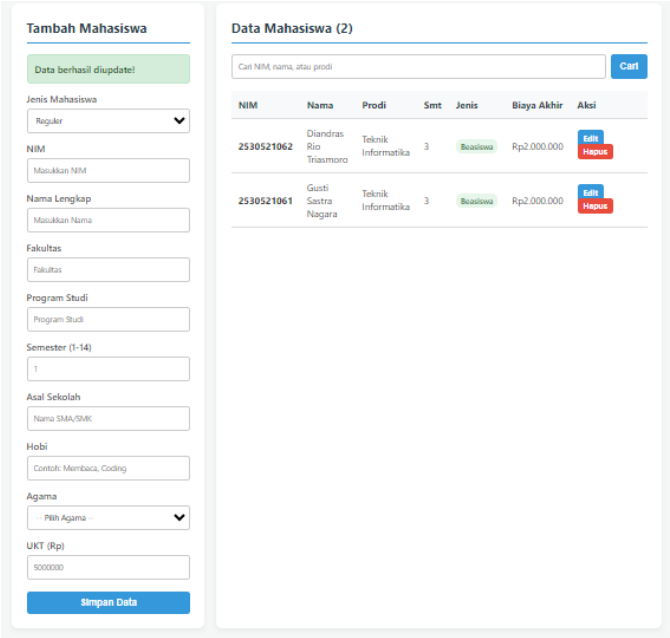
UKT -100



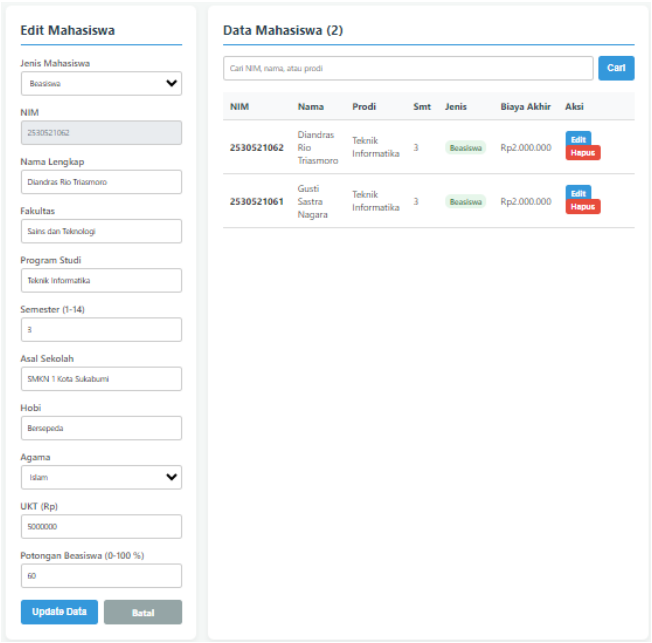
Potongan 120%



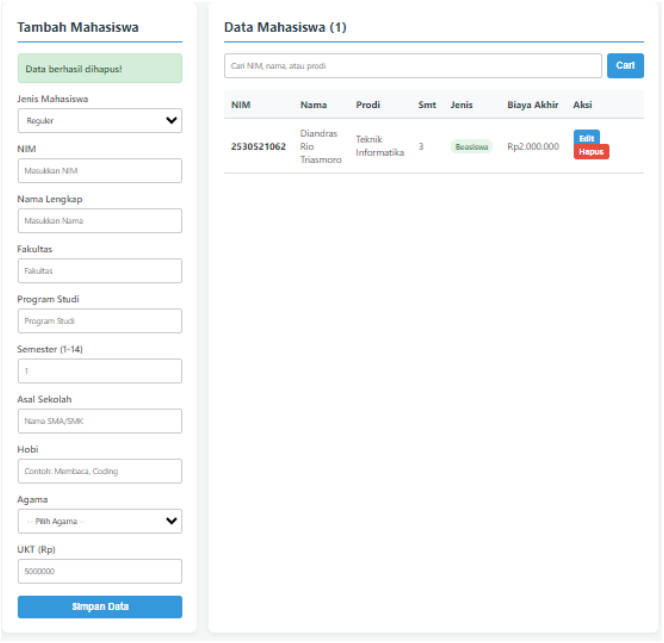
NIM duplikat



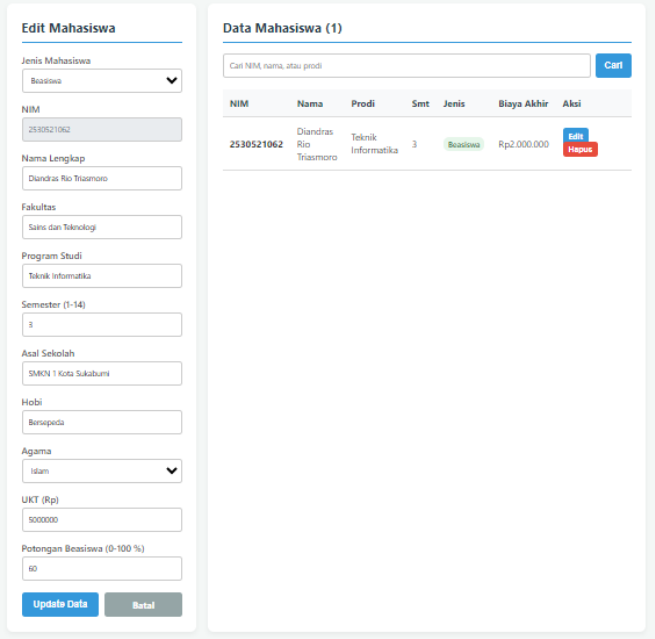
Edit Reguler menjadi Beasiswa pada NIM sama



Edit dengan NIM berbeda



Hapus satu record



Refresh browser selama Session aktif

Warning: Undefined property: MahasiswaReguler::$nim in **C:\xampp\htdocs\pbo06-inheritance\Tugas Praktikum\Tugas 3\uji_private.php** on line **4**

Akses $m->nim langsung

5. Tabel Hasil Pengujian (12 Skenario)

No	Skenario	Hasil yang diharapkan	Status
01	Tambah Reguler UKT 5.000.000	Biaya akhir 5.000.000	Sesuai
02	Tambah Beasiswa UKT 5.000.000; potongan 60%	Biaya akhir 2.000.000	Sesuai
03	NIM kosong	Ditolak parent constructor	Sesuai
04	Semester 0 atau 15	Ditolak parent constructor	Sesuai
05	UKT -100	Ditolak child constructor	Sesuai

06	Potongan 120%	Ditolak child constructor	Sesuai
07	NIM duplikat	Ditolak repository	Sesuai
08	Edit Reguler menjadi Beasiswa pada NIM sama	Jenis berubah; data tetap satu record	Sesuai
09	Edit dengan NIM berbeda	Ditolak repository	Sesuai
10	Hapus satu record	Record lain tetap tersedia	Sesuai
11	Refresh browser selama Session aktif	Data tetap ada	Sesuai
12	Akses $m->nim langsung	Ditolak karena private	Sesuai

6. Jawaban Pertanyaan Penguatan Konsep

1. Mengapa MahasiswaBeasiswa disebut is-a Mahasiswa, bukan has-a?

Karena mahasiswa beasiswa adalah salah satu jenis mahasiswa. Ia punya semua ciri mahasiswa (NIM, nama, prodi, semester) dan masih bisa diperlakukan sebagai mahasiswa. Has-a berarti sebuah class hanya memiliki atau memakai objek lain sebagai bagiannya, misalnya Mahasiswa memiliki Alamat.

2. Apa yang terjadi jika child membuat constructor sendiri tetapi lupa memanggil parent::__construct()?

Constructor parent tidak dijalankan, sehingga property parent (nim, nama, prodi, semester) tidak terisi dan validasinya terlewati. Karena property bertipe (typed), membaca property itu lewat getter akan menimbulkan error "must not be accessed before initialization".

3. Mengapa child tidak boleh langsung membaca private $nim pada parent?

Karena private hanya bisa diakses oleh class yang mendeklarasikannya. Ini menjaga enkapsulasi: perubahan data identitas harus lewat jalur terkontrol (constructor dan getter) agar validasi tidak terlewati. Child mengaksesnya lewat getNim().

4. Apa beda overriding dengan overloading di Pertemuan 4?

Overriding terjadi pada inheritance, ketika child mendefinisikan ulang method parent dengan nama dan signature yang kompatibel, sehingga perilakunya diganti.

Overloading di PHP berarti penanganan dinamis atas member yang tidak ada atau tidak dapat diakses lewat magic method seperti __call(), __get(), dan __set(). Ini bukan overloading seperti di Java atau C++.

5. **Mengapa hitungBiaya() dapat dipanggil dari variabel bertipe Mahasiswa?**

   Karena hitungBiaya() dideklarasikan (abstract) di Mahasiswa, dan setiap child wajib mengimplementasikannya. Objek child juga adalah Mahasiswa, jadi PHP menjalankan implementasi sesuai objek aslinya. Inilah dasar polimorfisme.

6. **Apakah use TraitX sama dengan extends ParentX?**

   Tidak. extends membentuk hubungan is-a dan hanya boleh satu parent. use trait hanya menyisipkan method dan property ke dalam class sebagai mekanisme reuse kode, tanpa hubungan is-a (objek tidak menjadi instanceof trait), dan sebuah class boleh memakai banyak trait. Jadi trait bukan multiple inheritance.

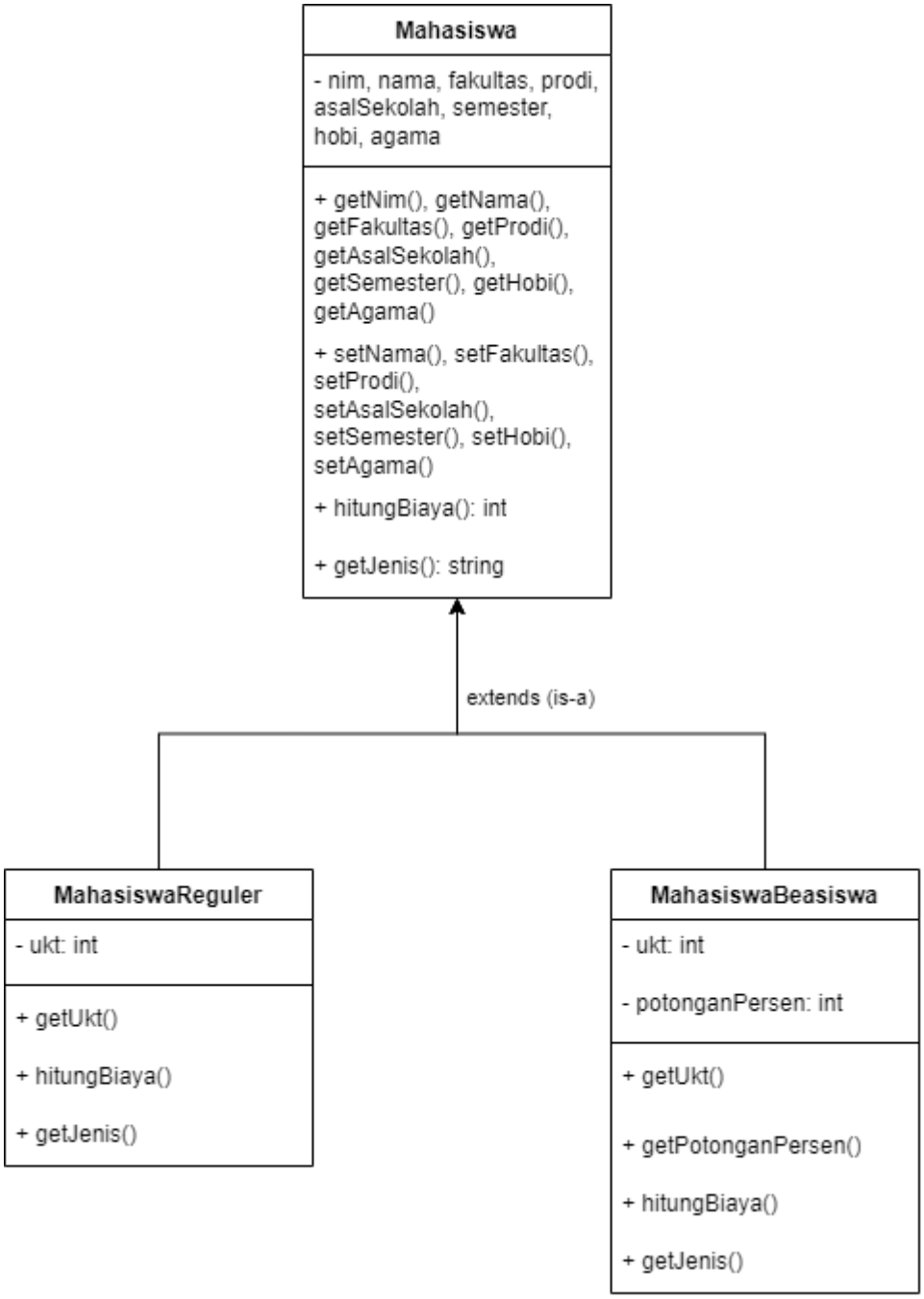
7. **Jawaban Tugas Praktikum**

   Tugas 1 — Diagram class dan penjelasan (individu)

   Gambarkan hubungan Mahasiswa, MahasiswaReguler, dan MahasiswaBeasiswa. Tandai property/method yang diletakkan di parent dan child. Jelaskan minimal tiga alasan pemilihan desain. Bandingkan private dan protected pada contoh nyata.

   Jawaban

   Diagram class



Pembagian parent dan child

- Di parent (Mahasiswa): data yang dimiliki semua mahasiswa (NIM, nama, fakultas, prodi, asal sekolah, semester, hobi, agama), getter/setter-nya, dan validasinya (NIM dan nama tidak kosong, semester 1-14, agama harus valid).
- Di child: hanya hal yang berbeda, yaitu ukt (kedua child), potonganPersen (Beasiswa), serta implementasi hitungBiaya() dan getJenis().

Alasan pemilihan desain

1. Hubungan is-a benar-benar berlaku. Mahasiswa Reguler adalah mahasiswa, begitu juga Mahasiswa Beasiswa. Keduanya punya identitas yang sama dan hanya berbeda pada aturan biaya.
2. Menghindari duplikasi kode. Delapan property identitas beserta validasinya cukup ditulis sekali di parent. Tanpa inheritance, semuanya harus disalin ke dua class dan setiap perubahan aturan harus diubah di dua tempat.
3. Repository dan tampilan tidak perlu percabangan. DataMahasiswa menerima parameter bertipe Mahasiswa, sehingga objek Reguler maupun Beasiswa bisa disimpan. Tabel cukup memanggil $m->hitungBiaya() tanpa if untuk mengecek jenisnya.

Private dan protected pada contoh nyata

- private hanya bisa diakses langsung oleh class yang mendeklarasikannya. Pada proyek ini nim adalah private di Mahasiswa. MahasiswaReguler tidak bisa menulis $this->nim, tetapi bisa memakai $this->getNim(). Dengan begitu aturan NIM (tidak kosong dan tidak bisa diubah setelah dibuat) tetap dijaga oleh parent.
- protected bisa diakses oleh class itu sendiri dan turunannya, tetapi tidak dari luar. Misalnya protected string $kategori di parent bisa dibaca langsung oleh child lewat $this->kategori.

Tugas 2 — Implementasi inheritance sederhana (individu)

Buat parent class Pegawai dengan id, nama, dan getNama(); turunkan PegawaiTetap dengan gajiPokok dan PegawaiKontrak dengan upahPerJam. Implementasikan hitungGaji() berbeda pada masing-masing child. Gunakan parent::__construct(), validasi angka tidak negatif, dan tampilkan output dua object.

Jawaban

Kode Program

```php
<?php
declare(strict_types=1);

abstract class Pegawai {
    public function __construct(private string $id, private string
$nama) {
        if (trim($id) === '' || trim($nama) === '') {
            throw new InvalidArgumentException('ID dan nama wajib
diisi');
        }
```

```
    }

    public function getId(): string { return $this->id; }
    public function getNama(): string { return $this->nama; }

    abstract public function hitungGaji(): int;
}

class PegawaiTetap extends Pegawai {
    public function __construct(string $id, string $nama, private int
$gajiPokok) {
        parent::__construct($id, $nama);
        if ($gajiPokok < 0) {
            throw new InvalidArgumentException('Gaji pokok tidak boleh
negatif');
        }
    }

    public function hitungGaji(): int {
        return $this->gajiPokok;
    }
}

class PegawaiKontrak extends Pegawai {
    public function __construct(string $id, string $nama,
            private int $upahPerJam, private int $jamKerja) {
        parent::__construct($id, $nama);
        if ($upahPerJam < 0 || $jamKerja < 0) {
            throw new InvalidArgumentException('Upah per jam dan jam
kerja tidak boleh negatif');
        }
    }

    public function hitungGaji(): int {
        return $this->upahPerJam * $this->jamKerja;
    }
}

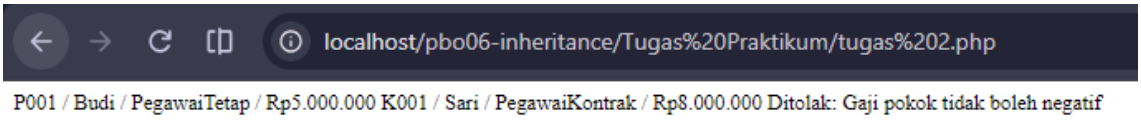
$daftar = [
    new PegawaiTetap('P001', 'Budi', 5000000),
    new PegawaiKontrak('K001', 'Sari', 50000, 160),
];

foreach ($daftar as $p) {
    echo $p->getId() . ' / ' . $p->getNama() . ' / ' . get_class($p)
        . ' / Rp' . number_format($p->hitungGaji(), 0, ',', '.') .
PHP_EOL;
}

// Uji validasi angka negatif
try {
    new PegawaiTetap('P002', 'Rudi', -1);
} catch (InvalidArgumentException $e) {
    echo 'Ditolak: ' . $e->getMessage() . PHP_EOL;
```

```
}
```

Hasil pengujian



Penjelasan singkat untuk laporan

- id, nama, dan getNama() ada di parent Pegawai, jadi tidak diulang di kedua child.
- Setiap child memanggil parent::__construct() untuk mengisi data umum, lalu memvalidasi angka miliknya sendiri (tidak boleh negatif).
- hitungGaji() diimplementasikan berbeda: pegawai tetap membayar gaji pokok, pegawai kontrak membayar upah per jam dikali jam kerja (50.000 x 160 = 8.000.000).
- Perulangan foreach memanggil hitungGaji() tanpa mengecek jenis objeknya.

Tugas 3 — Pengembangan CRUD Mahasiswa (kelompok)

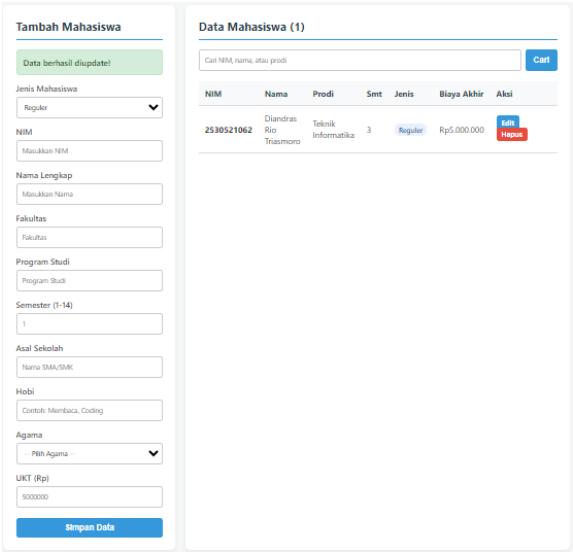
Lanjutkan CRUD Pertemuan 5: form harus dapat memilih Reguler/Beasiswa; data tambahan UKT dan potongan sesuai jenis; tabel menampilkan jenis dan biaya akhir; fitur tambah, cari, edit, dan hapus tetap berfungsi. Edit boleh mengubah jenis tetapi tidak boleh mengubah NIM. Tambahkan validasi UKT >= 0 dan potongan 0–100. Tidak diperbolehkan menggandakan kode NIM/nama/prodi/semester pada child.

Jawaban

Kode program

https://github.com/Diandras-Rio/pbo06-inheritance

Hasil Pengujian



Tugas 4 — Analisis inheritance vs komposisi (individu)

Tentukan apakah hubungan berikut lebih tepat is-a atau has-a, lalu berikan alasan: Mahasiswa– MahasiswaBeasiswa; Mahasiswa–Alamat; Mahasiswa–KartuPerpustakaan; Mahasiswa– MahasiswaReguler. Jelaskan satu contoh desain inheritance yang keliru dan perbaikannya.

Jawaban

Hubungan	Jenis	Alasan
Mahasiswa – MahasiswaBeasiswa	is-a	Mahasiswa beasiswa adalah jenis dari mahasiswa. Ia punya semua data dan perilaku mahasiswa, ditambah potongan biaya.
Mahasiswa – Alamat	has-a	Mahasiswa bukan jenis dari alamat. Alamat adalah data yang dimiliki mahasiswa, jadi dimodelkan sebagai objek Alamat di dalam Mahasiswa.
Mahasiswa – KartuPerpustakaan	has-a	Mahasiswa memiliki kartu perpustakaan, tetapi bukan jenis dari kartu. Kartu juga punya data sendiri (nomor, masa berlaku) dan bisa diganti tanpa mengubah mahasiswa.

Mahasiswa – MahasiswaReguler	is-a	Mahasiswa reguler adalah salah satu jenis mahasiswa dengan aturan biaya UKT penuh.

Contoh desain inheritance yang keliru dan perbaikannya

Desain yang keliru

```php
<?php
class Alamat {
    public function __construct(public string $jalan, public string
$kota) {}
}

class Mahasiswa extends Alamat {   // KELIRU: mahasiswa bukan sebuah
alamat
    public function __construct(string $jalan, string $kota, private
string $nama) {
        parent::__construct($jalan, $kota);
    }
}
```

Masalahnya

kalimat "Mahasiswa adalah Alamat" tidak masuk akal, sehingga uji is-a gagal. Mahasiswa juga ikut mewarisi hal yang tidak relevan dengannya, dan tidak bisa memiliki lebih dari satu alamat (misalnya alamat asal dan alamat kos) karena hanya ada satu parent.

Perbaikan dengan komposis

```php
<?php
class Alamat {
    public function __construct(private string $jalan, private string
$kota) {}
    public function getLengkap(): string { return $this->jalan . ', '
. $this->kota; }
}

class Mahasiswa {
    public function __construct(private string $nama, private Alamat
$alamat) {}
    public function getAlamat(): Alamat { return $this->alamat; }
}
```

Sekarang hubungannya benar: mahasiswa **memiliki** alamat. Kalau nanti dibutuhkan alamat kos, cukup menambah satu property Alamat lagi tanpa mengubah struktur class.

8. Kesimpulan

Dari praktikum ini saya memahami bahwa inheritance memungkinkan class turunan memakai kembali property dan method parent melalui extends, selama hubungannya benar-benar is-a. Pada CRUD mahasiswa, data umum seperti NIM, nama, prodi, dan

semester cukup ditulis sekali di Mahasiswa, sedangkan MahasiswaReguler dan MahasiswaBeasiswa hanya berisi hal yang berbeda, yaitu UKT dan potongan.

Constructor child harus memanggil parent::__construct() agar data parent terisi dan tervalidasi, dan property private parent hanya bisa diakses child lewat getter sehingga enkapsulasi tetap terjaga. Method hitungBiaya() dioverride dengan rumus berbeda di tiap child, jadi tabel cukup memanggil method yang sama tanpa memeriksa jenis objek. Saya juga belajar bahwa tidak semua hubungan cocok memakai inheritance; hubungan seperti Mahasiswa dan Alamat lebih tepat memakai komposisi (has-a).

9. Link Github

https://github.com/Diandras-Rio/pbo06-inheritance